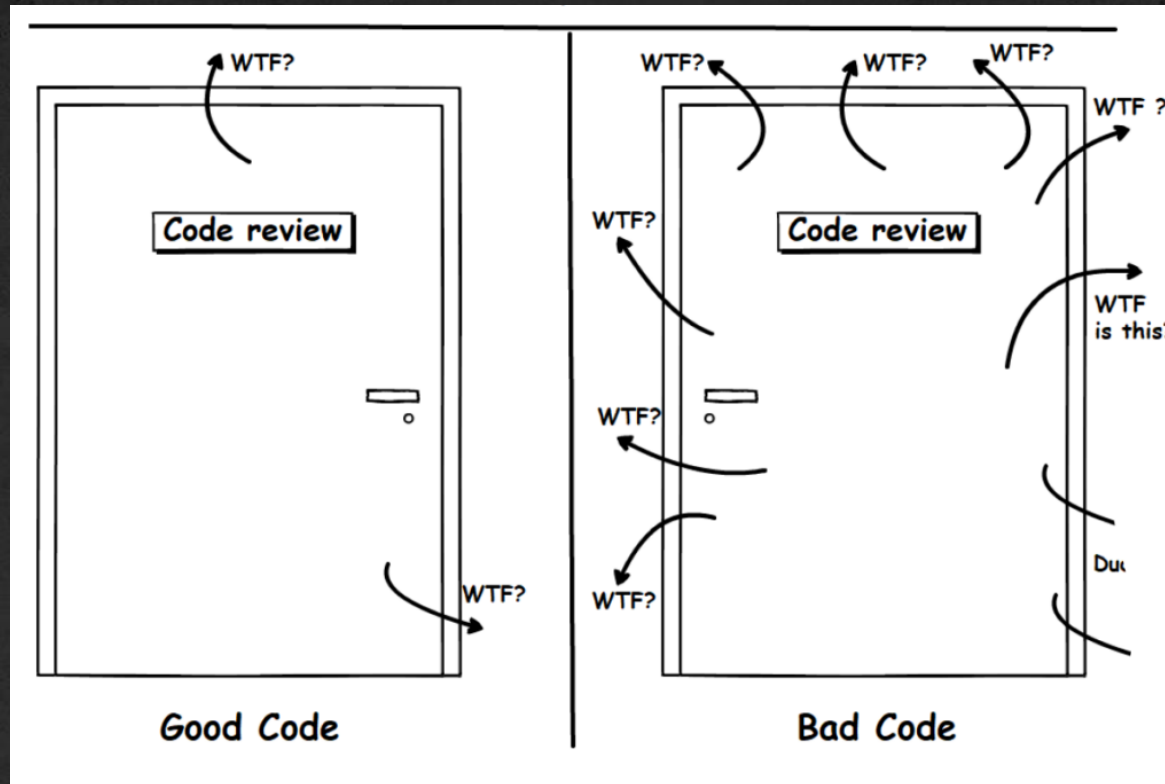




An introduction to Clean Code

Emilien RÉGENT
Senior Programmer at Ubisoft

Before Anything

Un code propre peut être

1. Compris facilement par tous les membres de l'équipe.
2. Amélioré par un développeur autre que son auteur.

Some Resources

1. En résumé
[Summary of 'Clean code' by Robert C. Martin](#)
2. Pour aller plus loin
[Clean Code by Uncle Bob](#)



Access Modifiers

- Limiter l'accès au strict nécessaire
- Préférer un champs privé sérialisé que public
- Pour les variables et les méthodes

```
// Wrong access modifier
public int m_MyCreditCardNumber = 7319;
public static bool IS_EVIL = true;

public void PurchaseWithNoConfirmation()
{
    // ...
}

// Correct access modifiers
private string m_MyPassword = "StaticIsEvil";
[SerializeField] private string[] m_ShoppingListOnTheFridge;

protected virtual void DoItYourOwnWay()
{
    // ...
}
```


Naming Conventions

- Noms clairs qui reflètent l'intention
- Pas d'abréviations
- Respecter une nomenclature au sein du projet

```
// Bad naming
private const int X = 42;
private void F(int a, int b)
{
    // ...
}

// Good naming
private const string PLAYER_TAG = "Player";
private bool m_IsPlayer = false;

private void SetPlayer()
{
    // ...
}

struct PlayerProfile
{
    // ...
}
```

Methods Structure

- Courte et concise
- Nombre réduit de paramètres
- Responsabilité unique

```
// Long method with unclear purpose
private void ProcessUserData(
    Object user, PlayerProfile player,
    string[] options, string[] settings)
{
    // ...
}

// Fetch player data from a profile
private string GetPlayerData(PlayerProfile playerProfile)
{
    return playerProfile.Name;
}
```


Single Responsibility

- Limiter les responsabilités
- Une classe
> un objet/concept
- Une méthode
> un changement

```
// Make the game
private void Update()
{
    GetComponent<MeshRenderer>().material.color = Color.blue;
    transform.position += transform.right * Time.deltaTime;
    Bird duck = new Bird();
    duck.NumberOfLegs = 3;
}

// One. At. A. Time.
private void Start()
{
    GetComponent<MeshRenderer>().material.color = Color.blue;
}

private void FixedUpdate()
{
    Move();
    CreateThreeLegsDuck();
}

private void Move()
{
    transform.position += transform.right * Time.deltaTime;
}

private void CreateThreeLegsDuck()
{
    Bird duck = new Bird();
    duck.NumberOfLegs = 3;
}
```

Code Duplication

- Extraire le code dupliqué
- Faciliter la réutilisation
- Faciliter la maintenabilité

```
// Duplicated code
class Car
{
    public int NumberOfKilometers { get; set; }
}

class Truck
{
    public int NumberOfKilometers { get; set; }
}

// Inheritance
public class Animal
{
    public int NumberOfLegs { get; set; }
}

public class Cat : Animal { }
public class Bird : Animal { }
```


Enforce Maintainability

- Exposer les paramètres d'équilibrage
- Pas de magic number (textes et nombres arbitraires)

```
// I'm a genius you can't understand
public float DoPreciseMath()
{
    return -9.80665f * 32f;
}

// Make it understable
private const float GRAVITY = -9.80665f;

[SerializeField] private float m_Speed = 32f;

public void ApplyGravity()
{
    transform.position += transform.up * GRAVITY * m_Speed * Time.deltaTime;
}
```

Comments

- In English!
- Uniquement quand nécessaire
- Préférez le code auto-documenté (aka nommage explicite)

```
private bool IsACat(Animal specimen)
{
    // Vérifie certains paramètres par sécurité
    if (specimen.NumberOfLegs == 4 && IS_EVIL)
    {
        // Retourne vrai
        return true;
        //return false;
    }

    // Check if specimen has every features of a Cat
    if (specimen.NumberOfLegs == 4 && IS_EVIL)
    {
        return true;
    }

    return false;
}
```


Let's Practice

```
19 private IEnumerator CookPizza() //IEnumerator fonction qui prend du temps et s'exécute sans bloquer les autres
20 {
21     //isCooking = true; // Empêche de relancer la cuisson plusieurs fois
22     dough.GetComponent<Rigidbody>().constraints = RigidbodyConstraints.FreezePosition;
23     yield return new WaitForSeconds(3); // Attend 3 secondes
24     Debug.Log("Pizza cuite !");
25     dough.GetComponent<Rigidbody>().constraints = RigidbodyConstraints.FreezeRotation;
26     dough.GetComponent<Renderer>().material = CookedDoughMaterial;
27     dough.tag = "CookedDough";
28     //isCooking = false; // Permet de relancer la cuisson si nécessaire
29 }
```

Let's Practice

```
24 void Update()
25 {
26     float distance = Vector3.Distance(player.position, transform.position);
27
28     if (distance <= detectionRange) // Si le joueur est dans le champ de vision
29     {
30         animator.SetFloat("Speed", 1); // Active l'animation de course
31         MoveTowardsPlayer(); // Déplacer Slender vers le joueur
32     }
33     else
34     {
35         animator.SetFloat("Speed", 0); // Désactive l'animation de course
36     }
37
38     // Vérifie si le joueur est à plus de 20 unités pendant 5 secondes
39     if (distance > teleportDistanceThreshold)
40     {
41         teleportTimer += Time.deltaTime;
42         if (teleportTimer >= teleportCooldown)
43         {
44             TeleportNearPlayer(); // Téléporte l'ennemi près du joueur
45             teleportTimer = 0f; // Réinitialise le chronomètre
46         }
47     }
48     else
49     {
50         teleportTimer = 0f; // Réinitialise le chronomètre si le joueur est proche
51     }
52 }
```


Let's Practice

```
5 public class JarInteraction : Interaction
6 {
7     [SerializeField] private GameObject ItemInJar;
8     [SerializeField] private GameObject debrisPrefab;
9     3 références
10    protected override void OnInteraction()
11    {
12        if (isPlayerClose)
13        {
14            if (ItemInJar != null)
15            {
16                ItemInJar.SetActive(true);
17            }
18            Instantiate(debrisPrefab, transform.position, transform.rotation);
19            gameObject.SetActive(false);
20        }
21    }
```

```
5 public class KeyInteraction : Interaction
6 {
7     [SerializeField] private int keyId;
8     3 références
9     protected override void OnInteraction()
10    {
11        if (isPlayerClose)
12        {
13            keyring.AddKey(keyId);
14            gameObject.SetActive(false);
15        }
16    }
```

Conclusion

